

Fault-Tolerant Multi-GPU LLM Serving with Adaptive KV-Cache Checkpointing

Progress Update -- 8B Model Results

Scope: 1B -> 8B Upgrade

What changed since last update

Dimension	slides_2 (1B)	This update (8B)
Model	Llama-3.2-1B-Instruct	Llama-3.1-8B-Instruct
Datasets	Synthetic (uniform length)	ShareGPT, CNN-DM, Alpaca (real)
Workloads	W1-W3 (single dataset)	W1_Chat + W4_Mixed (per-req SLO)
Duration	70s	300s (5 min)
SLO calibration	Manual constants	Auto calibration (calibrate.py)
Capacity model	Serial-time estimate	Decode-first (profile-based)

- E1a_Quick completed: **47/48 runs** (1 failed: Periodic-High/W1_Chat/Heavy/none)
- E2_Recovery completed: **8/8 runs**
- E3_Ablation pending

System Overview (Recap)

Two Core Components

- (1) **Benders-based Robust Scheduler** -- admission + routing, verified against failure scenarios
- (2) **Adaptive Checkpoint Policy** -- per-request, publish when: $D_{replay} > D_{load} + \lambda * D_{ckpt}$

Baselines

Baseline	Routing	Checkpoint	Role
No-FT	Greedy (FCFS)	None	Overhead-free ref (no recovery)
Periodic-Low	Greedy	Fixed every 10 blocks	Conservative checkpoint
Periodic-High	Greedy	Fixed every 1 block	"More ckpt = better?"
Our-System	Benders	Adaptive	Full system

Experiment Setup (8B)

Hardware & Model

- Llama-3.1-8B-Instruct, 2x NVIDIA RTX A6000 48GB (dp=2), 32GB checkpoint pool

Workloads

Workload	Dataset	TPOT SLO	Description
W1_Chat	ShareGPT (5000)	100ms	Chatbot, decode-heavy
W4_Mixed	50% Chat + 20% Alpaca + 30% CNN-DM	Per-request	Production mix

Load & Faults

- Load: Light (0.5 rps, 25%), Moderate (0.8 rps, 40%), Heavy (1.1 rps, 55%) -- calibrated to saturation
- Fault: none / F2_Mid (kill 1 GPU at t=150s via SIGKILL)
- Duration 300s, warmup 30s, seed=42

SLO Calibration (calibrate.py)

Procedure

- 1. **Find saturation RPS**: run No-FT, binary-search for RPS where TPOT p95 spikes (knee) -> ~2.0 rps
- 2. **Measure baselines at saturation**: TTFT p95 = 238.3ms, failover gap p95 = 1857.5ms
- 3. **Set SLO = multiplier x baseline**:
 - TTFT SLO = 5x baseline = 1192ms
 - Failover gap SLO = 3x baseline = 5572ms

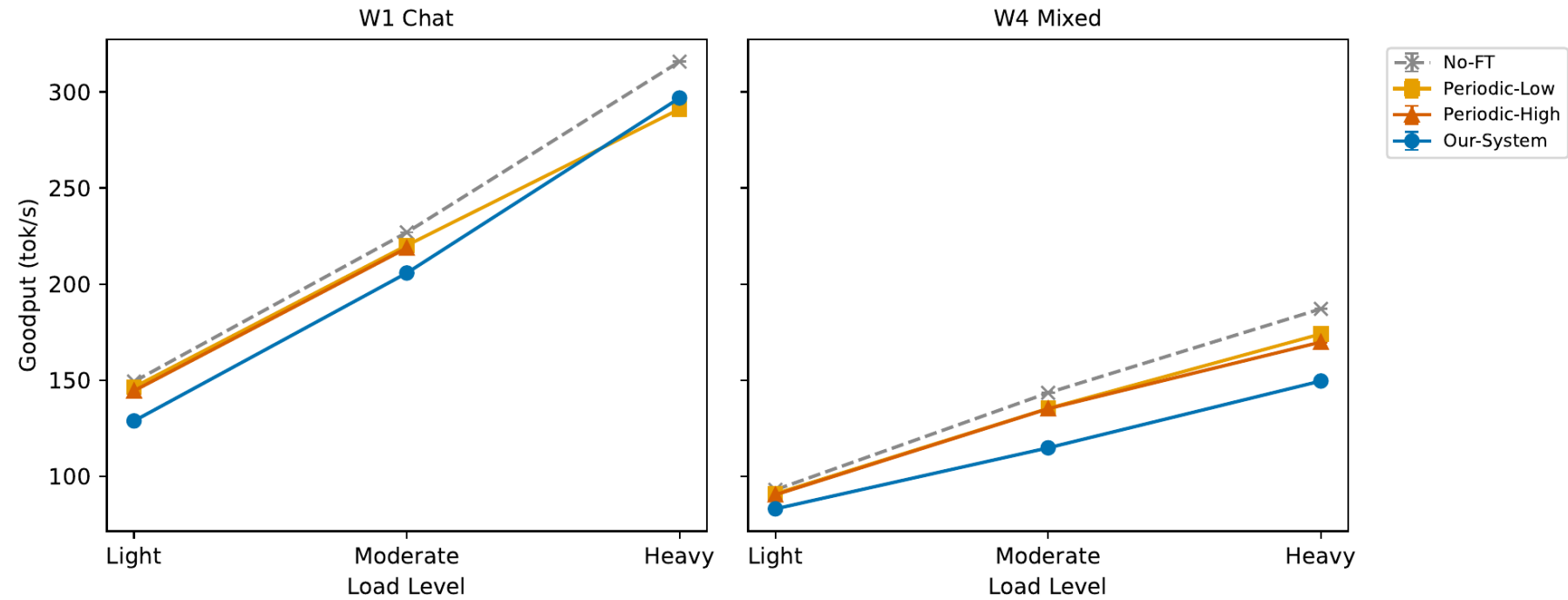
TPOT SLO (per-workload, application-driven, not calibrated)

Workload	TPOT SLO	Rationale
W1_Chat (chatbot)	100ms	Interactive, moderate latency tolerance
W3_Instruct (instruction)	50ms	Tight SLO, fast response expected
W2_Summary (summarization)	200ms	Tolerant, long output anyway
W4_Mixed	Per-request	Inherits from source workload

Why This Approach?

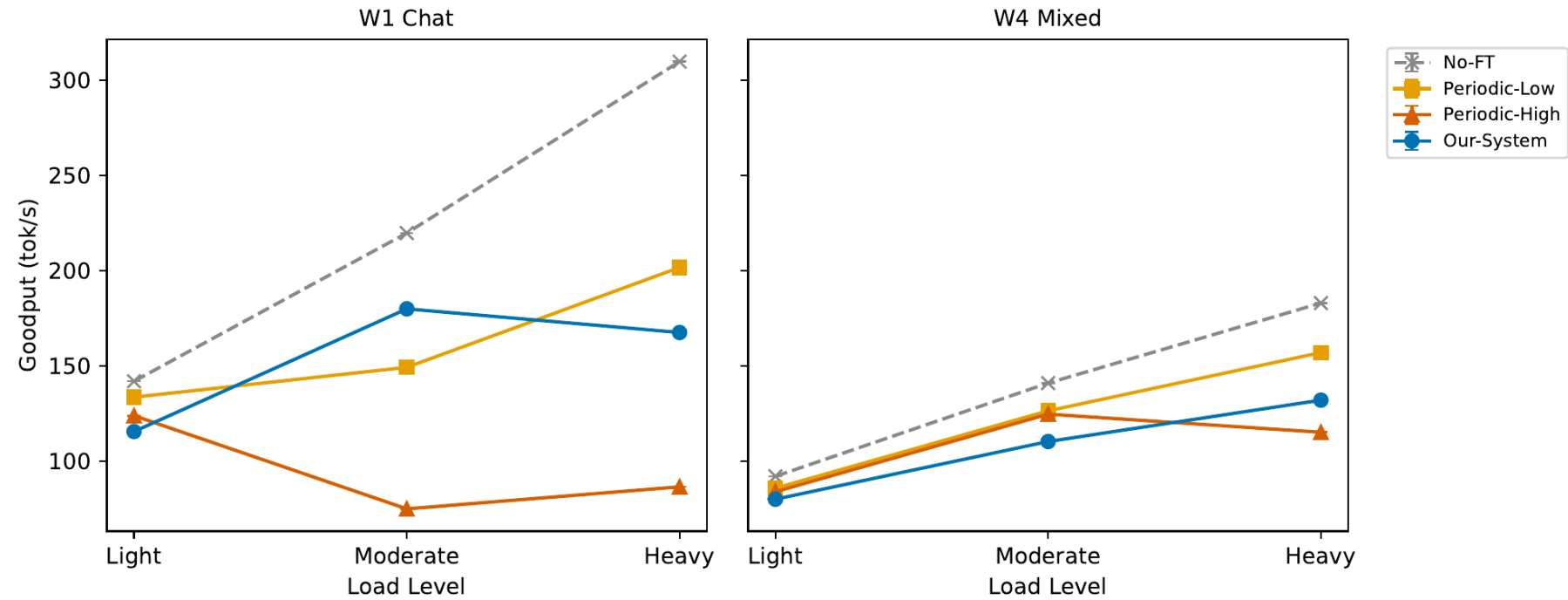
- TTFT/Gap SLO: hardware-dependent -> must calibrate per setup (auto, reproducible)
- TPOT SLO: application-dependent -> fixed by use case (follows related work: AdaServe, SOLA)

E1a: Goodput -- No Fault



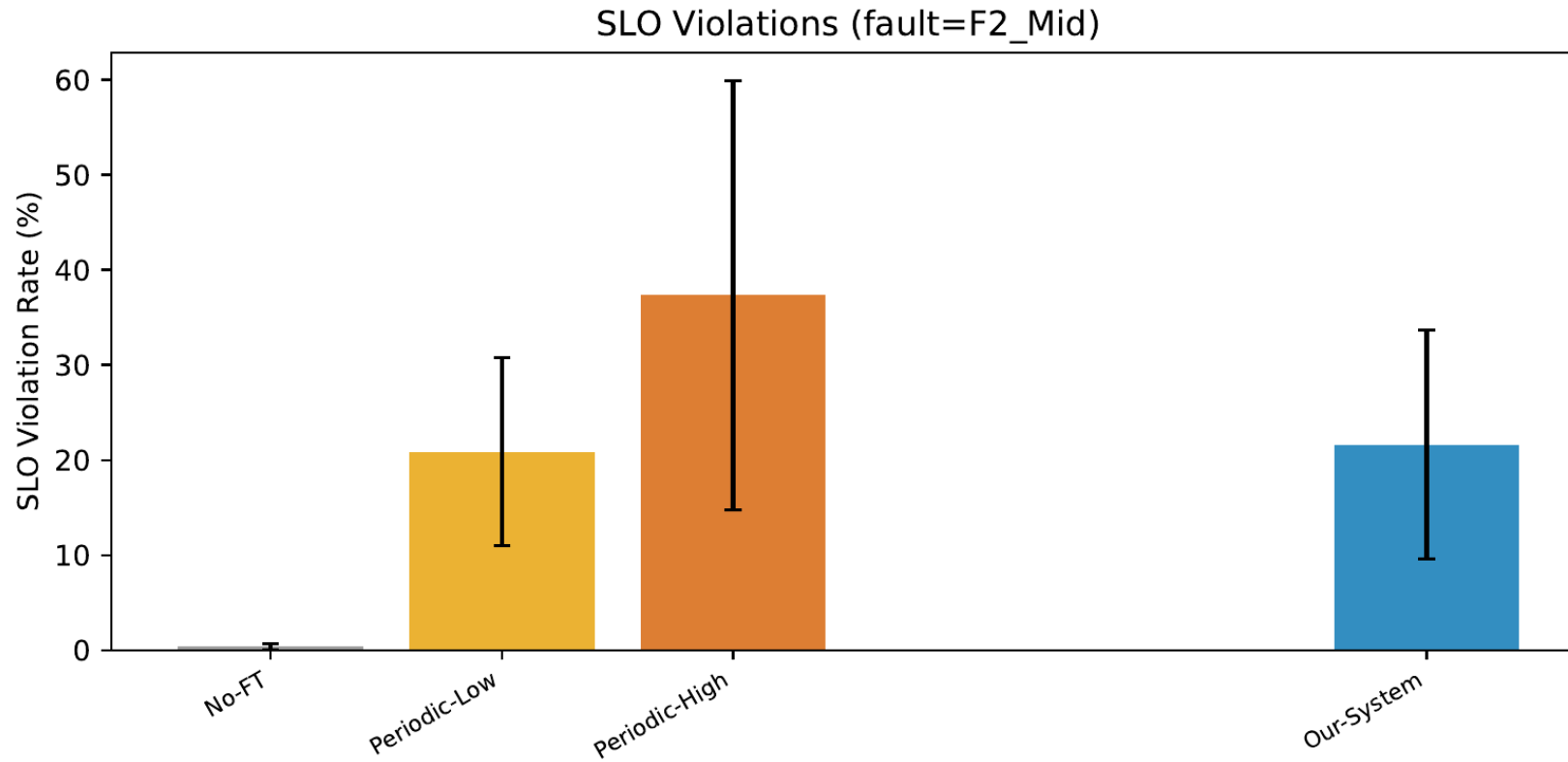
- **Periodic-Low overhead is small:** 2-8% below No-FT
- **Our-System overhead is 6-20%:** Benders solver + adaptive ckpt combined cost
- W4_Mixed shows higher overhead -- mixed SLOs make capacity planning harder
- 1 run failed: Periodic-High/W1_Chat/Heavy/none (max_model_len issue)

E1a: Goodput -- With Fault (F2_Mid)



- No-FT looks best but **drops requests silently** (0% recovery on W1_Chat)
- **Periodic-High collapses**: W1_Chat Mod 75 tok/s (-66% vs No-FT)
- **Our-System beats Periodic-High everywhere**, esp. W1_Chat Mod (180 vs 75, +140%)
- Our-System vs Periodic-Low: wins on Mod (180 vs 149), loses on Heavy (168 vs 202)

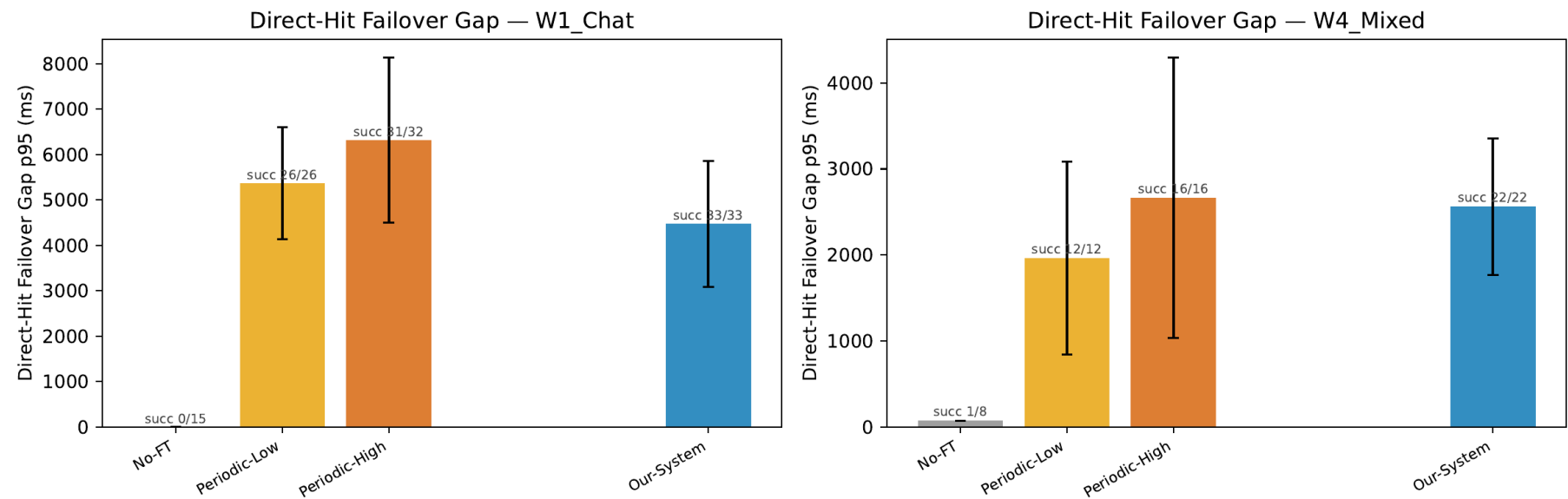
E1a: SLO Violation



Key Points

- No-FT ~0% violation (TPOT ~30ms, no FT work)
- **Our-System no-fault W4_Mixed: ~14%** -- Alpaca 50ms SLO is tight, solver overhead pushes TPOT to 75-115ms
- Under fault: Our-System **much better than Periodic-High** (23% vs 63% on W1_Chat Mod)
- SLO violation is the **main weakness** -- TPOT overhead from solver + checkpoint

E1a: Failover Gap & Recovery Rate



Recovery Rate (F2_Mid)

Baseline	W1 Light	W1 Mod	W1 Heavy	W4 Light	W4 Mod	W4 Heavy
No-FT	0%	0%	0%	0%	0%	25%
Periodic-Low	100%	100%	100%	100%	100%	100%
Periodic-High	100%	100%	94.1%	100%	100%	100%
Our-System	100%	100%	100%	100%	100%	100%

- **Our-System: 100% recovery everywhere** -- the only baseline to achieve this
- Periodic-High W1_Chat Heavy: 94.1% recovery + longest gap (8.3s)
- Our-System W1_Chat Heavy gap **3.4s vs Periodic-High 8.3s** (2.5x shorter)

E1a: Completion Rate

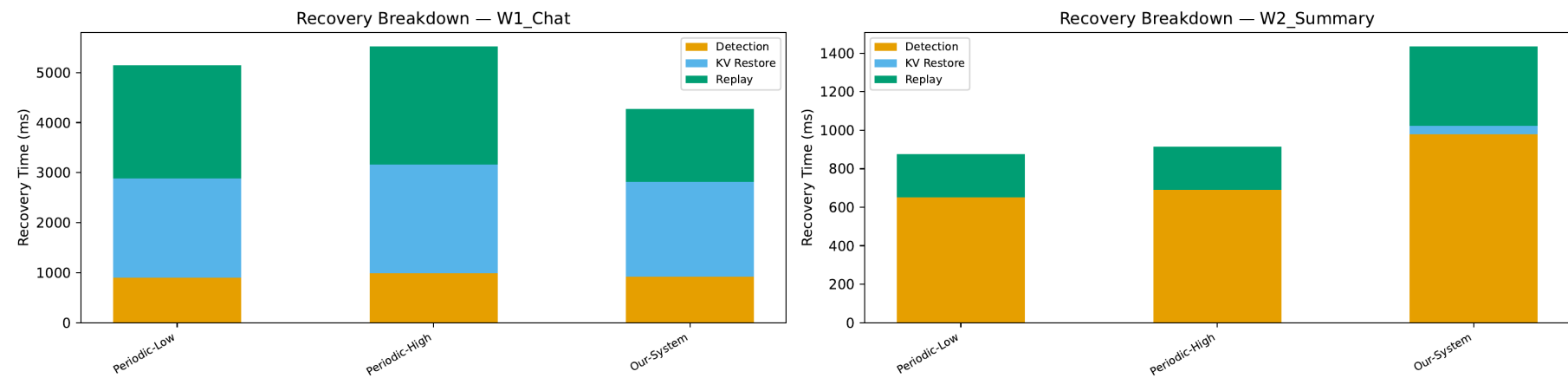
No Fault

Baseline	W1 Light	W1 Mod	W1 Heavy	W4 Light	W4 Mod	W4 Heavy
No-FT	100%	100%	100%	100%	100%	100%
Periodic-Low	100%	100%	100%	100%	100%	100%
Our-System	91.2%	95.6%	99.7%	98.1%	87.5%	89.6%

Key Points

- **Our-System completion < 100% even without faults** -- Benders solver over-rejects
- Worst: W4_Mixed Mod 87.5% (solver capacity model too conservative)
- Periodic-Low: 100% across the board -- greedy admission doesn't over-reject
- **Root cause:** decode-first capacity model underestimates true capacity

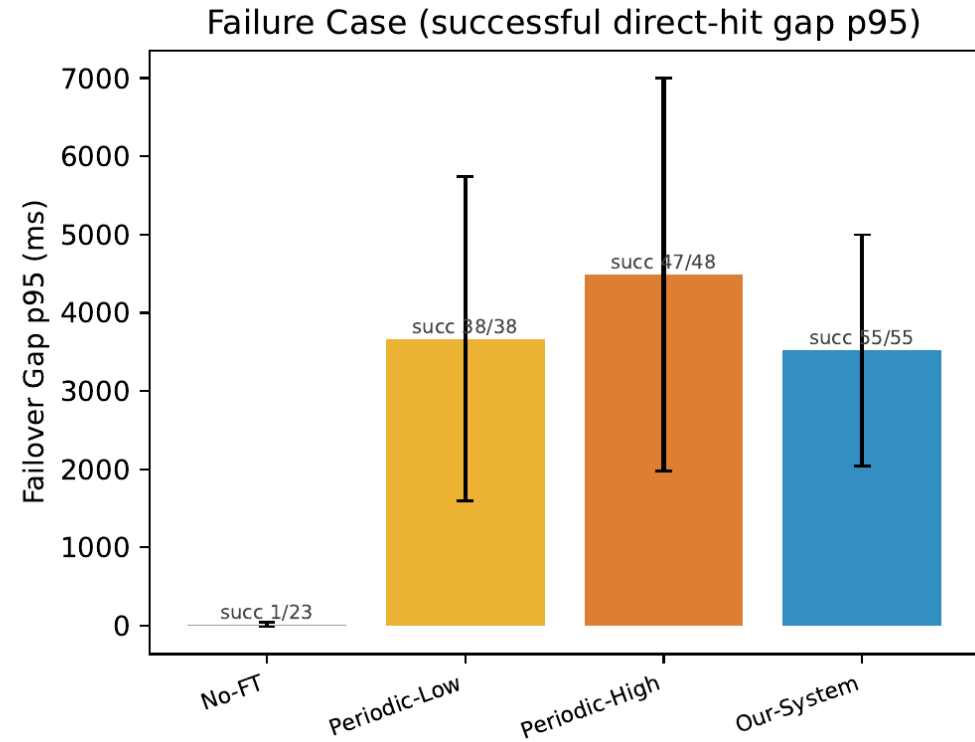
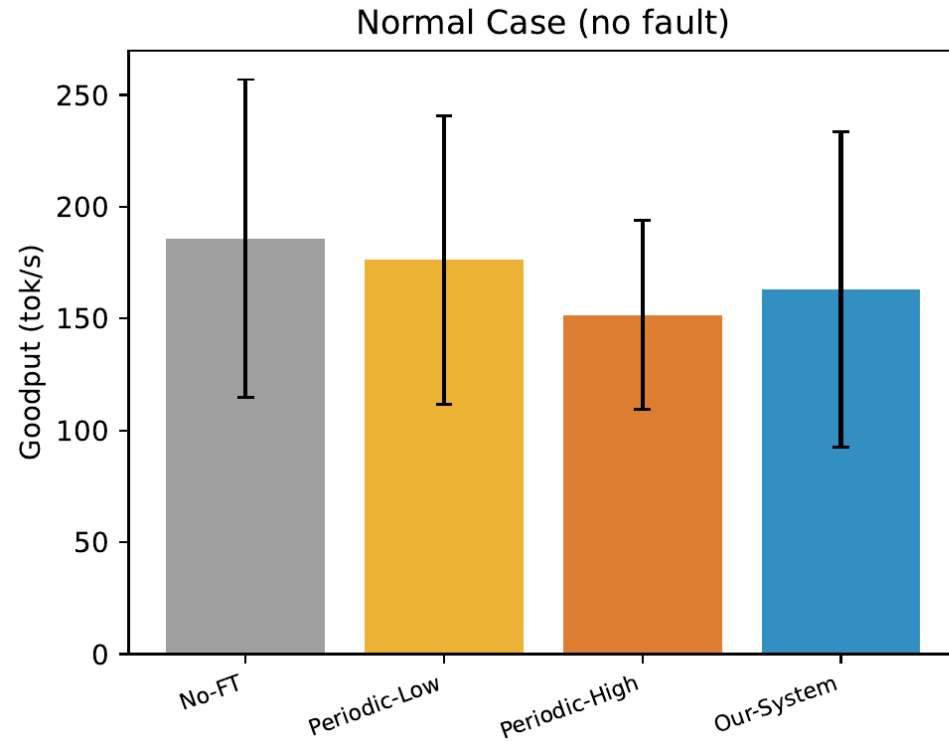
E2: Recovery Time Breakdown (8 runs, all F2_Mid, Moderate)



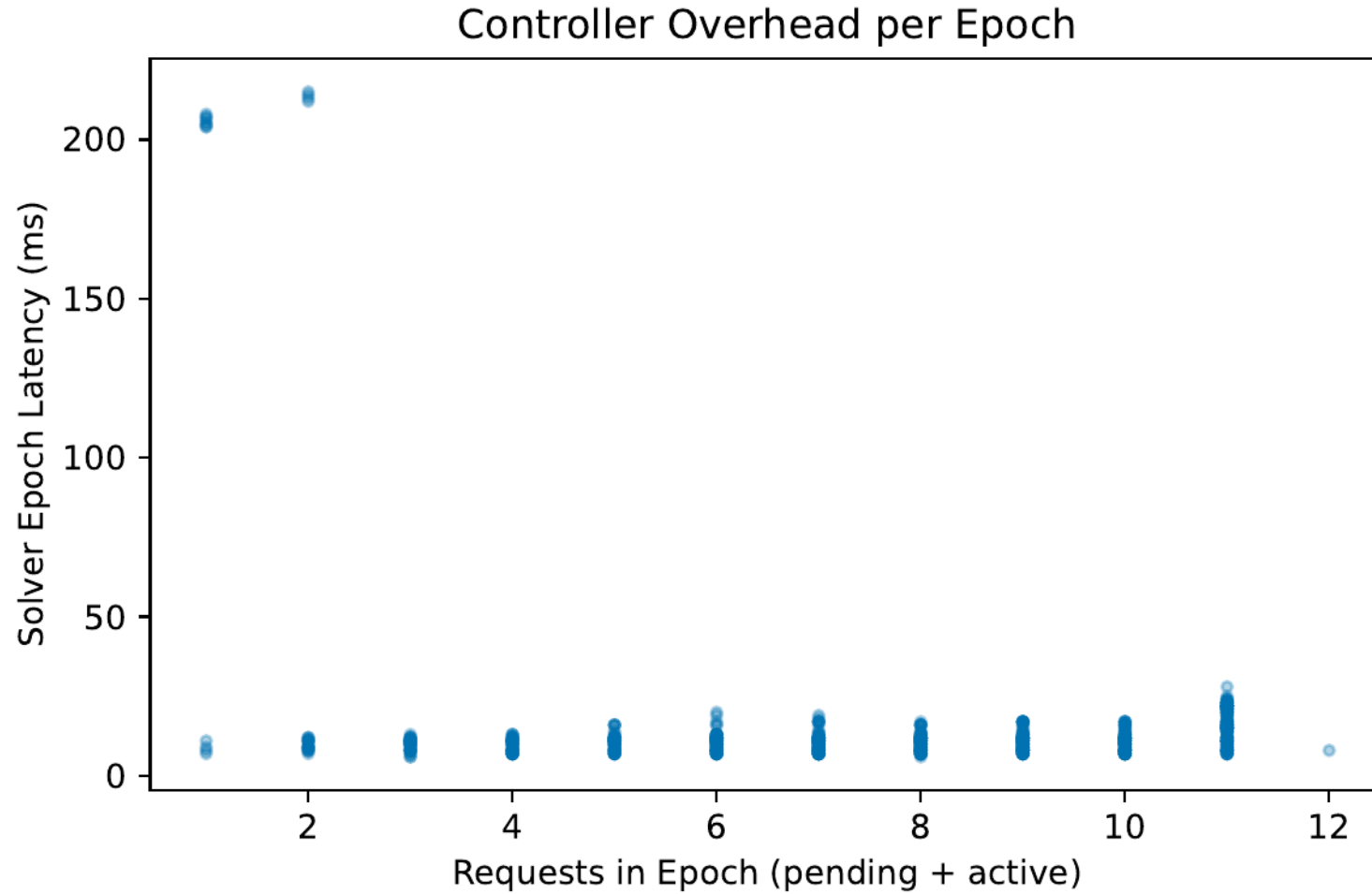
Baseline	Workload	Goodput	Compl	SLO Viol	TPOT p95	Gap p95
No-FT	W1_Chat	221.0	98.4%	0.0%	37.6ms	-- (0%)
No-FT	W2_Summary	55.0	99.6%	0.0%	31.1ms	-- (0%)
Periodic-Low	W1_Chat	151.4	100%	31.0%	124.3ms	6169ms
Periodic-Low	W2_Summary	55.1	100%	0.0%	84.0ms	1311ms
Periodic-High	W1_Chat	76.5	99.6%	62.8%	184.3ms	6768ms
Periodic-High	W2_Summary	55.1	100%	0.0%	93.1ms	471ms
Our-System	W1_Chat	162.0	91.9%	23.7%	118.8ms	5492ms
Our-System	W2_Summary	53.0	99.6%	2.8%	102.4ms	2170ms

- **Periodic-High W1_Chat worst:** goodput 76.5 (-65%), SLO viol 62.8%, gap 6768ms
- **Our-System W1_Chat:** best FT goodput (162), lowest SLO viol (23.7%), gap 5492ms
- **W2_Summary:** all FT baselines similar (~53-55) -- light workload, easy to handle

Checkpoint Overhead vs Recovery Benefit



- Normal-case overhead vs fault recovery benefit trade-off
- **Periodic-High pays most overhead but doesn't get best recovery**
- Our-System adaptive policy avoids unnecessary checkpoint work



- Solver latency per epoch across different request counts
- Solver runs **asynchronously** (fire-and-forget) -- doesn't directly add to TPOT
- Decisions have 1-epoch lag (~20ms), adding indirect scheduling delay

Summary: 8B Results

What Works

- 1. **100% recovery rate** -- only baseline to achieve this across all conditions
- 2. **Best goodput under fault on W1_Chat Mod** -- 180 vs Periodic-Low 149 vs Periodic-High 75
- 3. **Periodic-High is the worst FT strategy** -- validates "more ckpt != better"
- 4. **Failover gap competitive**: 3.4s vs Periodic-High 8.3s on Heavy

What Needs Improvement

- 1. **Normal-case overhead 6-20%** (was ~0% on 1B) -- solver + ckpt cost visible at 8B
- 2. **SLO violation 4-15% without faults** -- TPOT 75-115ms vs No-FT 30ms
- 3. **Completion < 100%** -- solver over-rejects (capacity model too conservative)

1B vs 8B Comparison

Metric	1B	8B
Normal-case overhead	~0%	6-20%
Recovery rate	100%	100%
SLO violation (no fault)	~0%	4-15%
Completion rate	88-100%	79-100%

Code Changes Since Last Update (1/2)

1. Performance Optimization -- 3 Async Pipelines

- **Problem:** FT overhead 22% in initial 8B test (Goodput 174.8 vs No-FT 223.1)
- **(a) Solver async** (ft_client.py): fire-and-forget, use previous epoch's result, epoch 100ms -> 20ms
 - Result: Goodput +7%, TTFT -11%
- **(b) Checkpoint copy async** (kv_checkpoint_pool.py): 2-stage pipeline
 - Stage 1: clone KV blocks to GPU buffer on default stream
 - Stage 2: async copy to pinned host memory on copy stream (record_event + wait_event)
- **(c) RPC fire-and-forget** (core.py): ThreadPoolExecutor.submit(), eager metadata update
 - Combined effect: Goodput 174.8 -> 215.6 (+23%), SLO violation 18.9% -> 4.9%

2. Decode-First Capacity Model

- Old: serial-time constraint overestimated prefill cost -> solver rejected feasible requests
- New: decode slot constraint + residual prefill capacity (matches vLLM's scheduling)
- Profile-based: profile_decode_capacity.py measures actual capacity at different batch sizes

Code Changes Since Last Update (2/2)

3. Bug Fixes

Bug	Impact	Fix
max_gpu_failures clamped to 0	Ckpt never published, solver infeasible	Removed clamping -- pass-through
EngineCore double admission	Solver admitted, engine re-rejected	register_admitted_request() for centralized
num_ckpt_tokens not aligned	Solver saw stale ckpt state	Let record_checkpoint set correct value
Objective penalty term	Penalized accepting requests	Removed penalty (not in design spec)

4. Real Datasets & Calibration

- 3 datasets: ShareGPT (chat), CNN-DailyMail (summary), Alpaca (instruction)
- Auto calibration (calibrate.py): binary-search saturation RPS + SLO baselines
- Per-request SLO: W4_Mixed assigns 50ms / 100ms / 200ms TPOT per dataset

5. Experiment Framework v2

- YAML-driven config (baselines, workloads, loads, faults, seeds)
- Auto server lifecycle + fault injection + metric collection
- 15 figure types (goodput, SLO, gap, ablation, heatmap, CDF, timeline, ...)

Next Steps

Priority	Task	Why
P1	Fix completion rate	Solver over-rejects -- tune capacity model
P1	Run E3_Ablation	Separate routing vs ckpt contribution
P2	Reduce SLO violation	TPOT 75-115ms is main gap vs No-FT
P3	Retry failed run	Periodic-High/W1_Chat/Heavy/none
P4	Scale to dp=4 (4 GPUs)	Routing value + realistic capacity loss (25%)
P5	Multiple seeds (3-5)	Mean +/- std for statistical significance